Progress Report 2: PM2.5 Prediction ML System

วิชา: ML Systems **หัวข้อโปรเจกต์:** FoonAlert – Bangkok PM2.5 Prediction & Alert System **วันที่:** มีนาคม 2026

---

1. Project Overview (ปรับปรุงจาก Progress 1)

FoonAlert คือระบบ Machine Learning สำหรับพยากรณ์ค่าความเข้มข้น PM2.5 (μg/m³) รายชั่วโมงล่วงหน้า **24 ชั่วโมง (forecast horizon = 24h)** โดยครอบคลุมทุกเขตในกรุงเทพมหานคร เพื่อตอบโจทย์ปัญหามลพิษทางอากาศที่ส่งผลกระทบโดยตรงต่อสุขภาพประชาชน โดยเฉพาะกลุ่มเปราะบาง เช่น ผู้สูงอายุ เด็กเล็ก และผู้ที่มีโรคทางเดินหายใจ

กลุ่มผู้ใช้งาน (Use Case):

- ประชาชนทั่วไป — วางแผนกิจกรรมกลางแจ้ง (ออกกำลังกาย, เดินทาง)
- หน่วยงานสาธารณสุข — ออกประกาศเตือนภัยล่วงหน้า
- นักพัฒนา / นักวิจัย — เชื่อมต่อ API เพื่อใช้ข้อมูลพยากรณ์

ประเภทของ ML Problem: Time-Series Regression — ทำนายค่าต่อเนื่อง (continuous value) จากข้อมูลในอดีต (hourly PM2.5, meteorological variables)

Dataset: ข้อมูลรายชั่วโมงจากเว็บ **AirBKK (https://official.airbkk.com)** ครอบคลุม **ม.ค. 2024 – มี.ค. 2026** รวม **86 สถานี** ในกรุงเทพมหานคร และ **13 ตัวแปร** (PM2.5, PM10, BP, Temp, RH, WS, WD, NO2, NO, NOX, O3, CO, RAIN) รวมทั้งหมด **~21.9 ล้าน rows** ใน combined dataset (`airbkk_all_stations_long_2023_to_latest.csv`) และไฟล์แยกต่อสถานี 53 ไฟล์ (`station_XX_long.csv`) โดยได้พัฒนา custom scraper ส่ง HTTP request ไปยัง AirBKK endpoint แบบ time-based query ซึ่งเป็นการขยายจาก Progress 1 ที่ใช้เฉพาะ PM2.5 รายวันของสถานีเดียว (10T) เป็น **multi-station, multi-variable, hourly dataset**

โมเดลที่เลือกใช้:

- **Random Forest** — **Champion model** (MAE 4.59 μg/m³) ให้ผลดีที่สุดในการทดสอบ เหนือกว่า LSTM ที่ MAE 6.22
- **LSTM (Long Short-Term Memory)** — train และทดสอบแล้ว แต่ด้วยข้อมูล tabular + lag features ที่ใช้ RF จับ short-term pattern ได้ดีกว่า LSTM ใน dataset นี้
- โมเดลทุกตัว export เป็น **ONNX** สำหรับ inference ที่เร็วขึ้น และจัดการผ่าน **MLflow** สำหรับ experiment tracking + model registry

ผลการทดสอบโมเดล (Test Set — Station 10T, 2025):

Model	MAE ↓	RMSE ↓	R^2 ↑
Linear Regression (Baseline)	5.1348	6.7493	0.7726
Ridge Regression	4.8286	6.5294	0.7871
Random Forest	**4.5869**	6.6809	0.7772
XGBoost	4.9735	7.3464	0.7305
LSTM (PyTorch)	6.2156	8.2195	0.6627

ความคืบหน้าล่าสุด:

- รวบรวมข้อมูล hourly ครบตั้งแต่ปี 2024–2026 ทุกเขตในกทม.
- Train และ evaluate ครบทุกโมเดล รวมถึง LSTM (PyTorch + skorch)
- Export ทุกโมเดลเป็น ONNX สำหรับ fast inference
- Deploy **FastAPI** service จริง (endpoints: `/predict`, `/actual`, `/retrain`)
- MLflow tracking server + Airflow DAGs implement และ deploy ด้วย Docker Compose
- Auto-retrain pipeline ทำงานจริง: MAE > 6.0 หรือ PSI > 0.2 → trigger retrain DAG

---

2. MLS Architecture and ML Pipeline Design

2.1 System Overview Diagram

ระบบแบ่งออกเป็น 2 เส้นทางหลัก คือ **Training Pipeline** และ **Inference Pipeline** โดยมีส่วนประกอบดังนี้:

```
╔═══════════════════════════════════════════════════════════╗
║                FoonAlert ML System Architecture                ║
╚═══════════════════════════════════════════════════════════╝

┌─────────────────────────────────────────────────────────┐
│                        DATA LAYER                       │
│                                                         │
│  AirBKK API / PCD ──► Raw Data Store ──► Data Validation │
│  (Hourly PM2.5,          (CSV/DB)        (range check,  │
│   Meteorological)                         missing flag) │
└─────────────────────────────────────────────────────────┘
                          │
                ┌─────────┴─────────┐
                │                   │
                ▼                   ▼

┌──────────────────────┐   ┌──────────────────────┐
│   TRAINING PIPELINE  │   │  INFERENCE PIPELINE  │
│  (Offline / Batch)   │   │  (Online / Periodic) │
│                      │   │                      │
│  Preprocessing       │   │  Feature Store       │
│  Feature Eng.        │   │  (latest 168h data)  │
│  Model Training      │   │  Load Model from     │
│  (LSTM / RF)         │   │  MLflow Registry     │
│  Hyperparameter      │   │  Predict next 24h    │
```

```
│  Tuning                  │          │  Add Confidence          │
│  Evaluation              │          │  Interval (PI)           │
│  MLflow Logging          │          │  Store Predictions       │
└──────────────────────────┘          └──────────────────────────┘
           │                                     │
           ▼                                     ▼

┌──────────────────────────┐          ┌──────────────────────────┐
│   MODEL REGISTRY         │          │   PREDICTION STORE       │
│   (MLflow)               │◄─────────│   + Alert Engine         │
│   Champion Model         │ promote  │   (PM2.5 > threshold     │
│   Challenger Model       │          │    → send alert)         │
└──────────────────────────┘          └──────────────────────────┘
           │
           ▼

┌──────────────────────────────────────────────────────────────────┐
│                       MONITORING LAYER                           │
│                                                                  │
│  Rolling MAE Tracker ──► Drift Detector ──► Retrain Trigger      │
│  (Airflow DAG)              (PSI / KS-test)    (MAE > threshold) │
└──────────────────────────────────────────────────────────────────┘
```

ความแตกต่างระหว่าง Training และ Inference:

ด้าน	Training Pipeline	Inference Pipeline
เวลาทำงาน	Offline / ตามกำหนด (retrain trigger)	Online / ทุกชั่วโมง
Input	ข้อมูลย้อนหลัง 2024–ปัจจุบัน	ข้อมูล 168 ชั่วโมงล่าสุด (7 วัน)
Output	Trained model artifact + metrics	PM2.5 prediction + prediction interval (24h ahead)
Compute	Heavy (GPU/multi-core สำหรับ LSTM)	Light (load model → forward pass)
Latency	ไม่สำคัญ (รัน background)	< 1 วินาทีต่อสถานี
Logging	MLflow experiments log	Prediction store (DB)

2.2 Training และ Inference Design

วิธีแบ่งข้อมูล (Temporal Split):

เนื่องจากข้อมูลเป็น time-series จึงใช้ **temporal split อย่างเคร่งครัด** ห้ามใช้ random split เพราะจะทำให้เกิด data leakage จากอนาคต:

ชุดข้อมูล	ช่วงเวลา	หมายเหตุ
Training	ม.ค. 2024 – ธ.ค. 2024	~8,760 ชั่วโมง × ทุกสถานี
Validation	ม.ค. 2025 – ธ.ค. 2025	ใช้ tune hyperparameters + early stopping
Test	ม.ค. 2026 – มี.ค. 2026	ใช้ประเมินผลสุดท้าย (held-out)

สำหรับ LSTM ใช้ **sliding window** ขนาด 168 ชั่วโมง (7 วัน) เป็น input sequence เพื่อทำนาย 24 ชั่วโมงข้างหน้า

ผลสรุปเปรียบเทียบโมเดล (Test Set — Station 10T):

Model	MAE ↓	RMSE ↓	R^2 ↑	หมายเหตุ
Linear Regression	5.1348	6.7493	0.7726	Baseline 2
Ridge Regression	4.8286	6.5294	0.7871	—
Random Forest	**4.5869**	6.6809	0.7772	Champion (API default)
XGBoost	4.9735	7.3464	0.7305	—
LSTM (PyTorch)	6.2156	8.2195	0.6627	Worse than RF on this dataset (New update from progress 2)

> **หมายเหตุ:** LSTM ให้ผลแย่กว่า Random Forest เนื่องจาก input sequence ใช้ 17 tabular features (lag, rolling, time) ที่ Random Forest สามารถจับ short-term pattern ได้ดีกว่า ใน dataset รายวัน 1 สถานี LSTM มีข้อได้เปรียบน้อยกว่าที่คาด อาจต้องการข้อมูล multi-station หรือ raw hourly sequence เพื่อดึงศักยภาพออกมา

แนวทางประเมินผล:

- Primary metric: **MAE** (μg/m³) — interpretable, หน่วยเดียวกับ PM2.5
- Secondary metrics: RMSE, R^2
- Regime-specific evaluation: แยก MAE ตามช่วงมลพิษ
  - **Low:** < 35 μg/m³ (Good / Moderate)
  - **Moderate:** 35–75 μg/m³ (Unhealthy for sensitive groups)
  - **High:** > 75 μg/m³ (Unhealthy / Very Unhealthy) — **สำคัญที่สุด**
- **95th percentile error** สำหรับ worst-case analysis

ระบบ Prediction Mode:

ระบบออกแบบเป็น **Batch Prediction (Periodic Online)** — รัน inference ทุก 1 ชั่วโมง โดยใช้ข้อมูล real-time ล่าสุด เหตุผล:

1. PM2.5 เปลี่ยนแปลงช้า ไม่ต้องการ sub-second latency
2. การทำนาย 24 ชั่วโมงล่วงหน้าเหมาะกับการ pre-compute และ cache
3. ลด compute cost เมื่อเทียบกับ real-time streaming

---

3. ML Lifecycle

3.1 วงจรชีวิตโมเดลในปัจจุบัน

```
[Data Collection]──►[Preprocessing]──►[Feature Eng.]──►[Training]
       ▲                                                    │
       │                                                    ▼
[Monitoring] ◄──[Deployment]◄──[Evaluation]◄──[Model Registry]
       │
       └──► MAE / PSI > Threshold? ──YES──► [Retrain Trigger]
```

ขั้นตอนที่ยังทำแบบ Manual:

- การสร้าง feature engineering ใหม่ (manual analysis + code update)
- การตัดสินใจ promote champion model (human review ก่อน deploy)
- การตรวจสอบ data quality anomaly (sensor malfunction detection)
- การปรับ retrain threshold เมื่อมีการเปลี่ยน deployment condition

ขั้นตอนที่ Automate แล้ว (implement จริงใน codebase):

- **Automated Training Pipeline:** `pm25_training_dag.py` — Airflow DAG ที่รัน feature engineering → train ทุกโมเดลแบบ parallel → evaluate → export ONNX
- **Automated Monitoring + Retraining:** `pm25_pipeline_dag.py` — รันทุกวัน 01:00 UTC ตรวจ rolling MAE (30 วัน) และ PSI; ถ้า threshold เกิน → trigger training DAG อัตโนมัติ
- **Log Management:** หลัง retrain → ลบ prediction/actual logs เพื่อป้องกัน stale data re-trigger retrain ซ้ำ
- **API Serving:** FastAPI (`src/api.py`) รับ `/predict`, `/actual`, `/retrain` requests พร้อม log ทุก prediction+actual ลงไฟล์

แนวทาง Automate ในอนาคตเพิ่มเติม:

- **Automated Model Promotion:** ถ้า challenger model มี MAE ต่ำกว่า champion ≥ 5% บน validation window → promote อัตโนมัติ (ปัจจุบันยังต้อง human review)
  - ตัวเลข MAE / threshold อาจจะปรับเปลี่ยนในอนาคตได้
- **Multi-station Expansion:** ขยาย pipeline ให้รองรับทุกสถานีในกทม. แบบ parallel

แนวทาง Retrain โมเดล:

เงื่อนไข	Action
Rolling MAE (30d) >**6.0 μg/m³**	Trigger `pm25_training_pipeline` DAG อัตโนมัติ
PSI >**0.2** (distribution shift)	Trigger retrain + alert
PSI 0.1–0.2	Monitor (yellow alert) — ยังไม่ retrain

เงื่อนไข	Action
เวลาผ่านไป 3 เดือน	Scheduled retrain (proactive, manual trigger)
พบ sensor anomaly หรือ data gap	Manual review → data remediation → retrain

Retrain Strategy: ใช้ **expanding window** — ไม่ทิ้งข้อมูลเก่า แต่เพิ่มข้อมูลใหม่เข้าไปเรื่อยๆ เพราะ PM2.5 มี long-term trend และ seasonality ที่สำคัญ

---

4. Infrastructure และ Tooling

4.1 สภาพแวดล้อมและเครื่องมือที่ใช้

ด้าน	เครื่องมือ / แนวทาง	หมายเหตุ
ภาษา	Python 3.11 (Docker) / 3.14 (local)	—
ML Framework	PyTorch + skorch (LSTM), scikit-learn (RF/Ridge/LR)	skorch เชื่อม PyTorch กับ sklearn API
Model Export	ONNX + onnxruntime	export ทุกโมเดลเป็น `.onnx` สำหรับ fast inference
API Serving	FastAPI + uvicorn	`src/api.py` — `/predict`, `/actual`, `/retrain`
Experiment Tracking	MLflow (PostgreSQL backend)	log params, metrics, artifacts, model registry
Pipeline Orchestration	Apache Airflow (LocalExecutor)	2 DAGs: training + monitoring/retrain
Infrastructure	Docker Compose	5 services: Postgres, MLflow, Airflow init/webserver/scheduler, API
Database	PostgreSQL 15	Airflow metadata + MLflow backend store
Data Storage	CSV/Parquet (shared volume)	Parquet สำหรับ inter-task data sharing ใน Airflow
Model Serialization	`.joblib` (sklearn), `.pt` (PyTorch), `.onnx`	save/load ใน inference pipeline
Version Control	Git + GitHub	https://github.com/yoaperm/pm25-prediction-ml-system

ด้าน	เครื่องมือ / แนวทาง	หมายเหตุ
Configuration	`configs/config.yaml`	กำหนด params ทั้งหมดที่จุดเดียว รวม monitoring thresholds
Testing	pytest (`tests/`)	unit tests สำหรับ preprocessing

4.2 การจัดเก็บข้อมูล

```
data/
├── raw/
│   ├── airbkk_all_stations_long_2023_to_latest.csv  # Combined ~21.9M rows, 86
stations, 13 params
│   ├── station_62_long.csv   # สถานีเดี่ยว long format (~325K rows)
│   ├── station_63_long.csv
│   ├── ... (53 ไฟล์, station 62–145)
│   └── PM2.5(2024).xlsx / PM2.5(2025).xlsx  # Legacy daily data (Progress 1)
└── processed/    # Parquet files ที่ Airflow tasks share กัน
    ├── train_features.parquet
    └── test_features.parquet

results/
├── experiment_results.csv   # Model comparison metrics
├── predictions_log.csv      # ทุก prediction ที่ส่งผ่าน /predict
├── actuals_log.csv          # ทุก actual ที่ส่งผ่าน /actual
└── monitoring_results.csv   # ผลทุก monitoring run
```

การ Scrape ข้อมูล: ได้พัฒนา custom scraper ส่ง HTTP request ไปยัง AirBKK endpoint แบบ time-based query เพื่อดึงข้อมูลรายชั่วโมงย้อนหลังตั้งแต่ปี 2024 ครอบคลุมทุกสถานีในกทม.

ขนาด Dataset ที่ได้:

ไฟล์	รายละเอียด	ขนาด
`airbkk_all_stations_long_2023_to_latest.csv`	Combined dataset ทุกสถานีในรูปแบบ long format	**~21.9 ล้าน rows**
`station_XX_long.csv` (53 ไฟล์)	ข้อมูลรายสถานี ตัวอย่าง:`station_62_long.csv` ~325K rows	รวม ~17.3 ล้าน rows

โครงสร้างข้อมูล (Long Format):

คอลัมน์	ตัวอย่างค่า	หมายเหตุ
`Date_Time`	`2024-01-01 00:00:00`	รายชั่วโมง

คอลัมน์	ตัวอย่างค่า	หมายเหตุ
`StationID`	62 – 145	86 สถานีในกทม.
`StationName`	เขตสัมพันธวงศ์	ชื่อเขต
`Parameter`	PM2.5, BP, RH, Temp, WS, WD, PM10, NO2, O3, CO, NO, NOX, RAIN	13 ตัวแปร
`Value`	46.3	ค่าตัวเลข

ช่วงเวลา: ม.ค. 2024 – มี.ค. 2026 (~3 ปี, ~19,500 ชั่วโมงต่อสถานี)

> **นัยสำคัญสำหรับ Modeling:** การมีตัวแปร meteorological (Temp, RH, WS, WD, BP) ครบในชุดข้อมูลเดียวกัน ทำให้สามารถเพิ่ม feature เหล่านี้ใน training phase ถัดไปได้โดยไม่ต้อง integrate external API แยก ซึ่งเป็นการขจัด limitation ที่ระบุไว้ใน Progress 1

4.3 ทรัพยากร Training

โมเดล	Hardware	เวลา Training (ประมาณ)
LSTM (PyTorch)	MacBook Pro / GPU (ถ้ามี)	~30–60 นาที (5 epochs, full dataset)
Random Forest	CPU (local)	< 5 นาที
Ridge Regression	CPU (local)	< 1 นาที

4.4 Docker Compose Stack

ระบบ deploy ผ่าน Docker Compose ซึ่งรัน 5 services พร้อมกัน:

```
┌──────────────────────────────────────────────┐
│              Docker Compose Stack            │
│                                              │
│  postgres:5432       ← Airflow metadata + MLflow DB │
│  mlflow:5000         ← Experiment tracking server   │
│  airflow-webserver:8080  ← DAG management UI        │
│  airflow-scheduler   ← DAG executor                 │
│  api:8000            ← FastAPI prediction service   │
└──────────────────────────────────────────────┘
```

```
# Start full stack
docker compose up -d
```

Service	URL	ใช้งาน
Airflow UI	http://localhost:8080	จัดการ DAG, trigger training/monitoring
MLflow UI	http://localhost:5001	เปรียบเทียบ experiments, ดู metrics
Prediction API	http://localhost:8001	`/predict` `/actual` `/retrain`
API Docs	http://localhost:8001/docs	Swagger UI

4.5 การบันทึกผลการทดลอง (MLflow)

MLflow ถูกใช้สำหรับ log:

- **Parameters:** learning_rate, n_epochs, window_size, hidden_size, dropout, n_estimators ฯลฯ
- **Metrics:** MAE, RMSE, R^2 (per model per run)
- **Artifacts:** model checkpoint, feature_columns.json
- **Comparison:** เปรียบเทียบ runs ทุกโมเดลผ่าน MLflow UI เพื่อเลือก champion

ทุก training run ถูก log ผ่าน `pm25_training_dag.py` โดยอัตโนมัติ

---

5. Data Shift และ Monitoring

5.1 ความเป็นไปได้ของ Data Distribution Shift

PM2.5 เป็นปรากฏการณ์ที่ได้รับอิทธิพลจากหลายปัจจัย ทำให้มีโอกาสเกิด distribution shift สูง:

ประเภท Shift	ตัวอย่าง	ผลกระทบต่อโมเดล
Concept Drift	เปลี่ยนแปลงนโยบายการจราจร, โรงงานใหม่, ไฟป่า El Niño	Pattern เปลี่ยน → model underperform
Covariate Shift	สภาพอากาศผิดปกติ (ลมพัดแรงผิดฤดูกาล)	Input distribution ต่างจาก training
Seasonal Shift	PM2.5 สูงขึ้นผิดปกติในฤดูแล้ง 2025 vs 2024	Rolling statistics ไม่ reprensent ปัจจุบัน
Sensor Drift	เซ็นเซอร์เสื่อม / calibration off	Label noise สะสม

5.2 Metrics ที่ควรติดตาม

Model Performance Metrics:

- `rolling_MAE_30d` — MAE เฉลี่ย **30 วัน**ล่าสุด (primary monitoring signal, threshold = 6.0 μg/m³)
- `rolling_MAE_high_regime` — MAE เฉพาะช่วง PM2.5 > 75 μg/m³
- `prediction_bias` — mean(predicted - actual) ตรวจว่าโมเดล over/under-estimate

Data Quality Metrics:

- `missing_rate_hourly` — % ข้อมูลหายต่อชั่วโมง
- `sensor_anomaly_rate` — จำนวนจุดที่ค่าผิดปกติ (< 0 หรือ > 500)
- `PSI (Population Stability Index)` — วัด distribution shift ของ input features

Drift Detection (implement แล้วใน `src/monitor.py`):

- **PSI (Population Stability Index)** เปรียบเทียบ distribution ของ predicted vs actual
  - PSI < 0.1 → stable (ปกติ)
  - PSI 0.1–0.2 → moderate shift (monitor)
  - PSI > 0.2 → significant shift → **trigger retrain**
- การตรวจสอบ PSI + MAE ทำงานผ่าน `pm25_pipeline_dag.py` ทุกวัน 01:00 UTC โดยต้องมีคู่ข้อมูล prediction+actual อย่างน้อย **7 pairs** จึงจะประเมินผล

5.3 แนวทางเบื้องต้นเมื่อ Performance ลดลง

```
Performance Drop Detected (rolling MAE > threshold)
         │
         ▼
   [Diagnose Root Cause]
         │
    ┌────┴────┐
    │         │
    ▼         ▼
Data Issue  Model Issue
(sensor,    (concept drift,
 missing)    new pattern)
    │         │
    ▼         ▼
Fix data    Trigger Retrain
remediation  with recent data
             │
             ▼
      Evaluate Challenger
      vs Champion Model
             │
             ▼
      Promote if better
      (MAE improvement ≥ 5%)
```

Threshold ที่ใช้จริง (ใน `configs/config.yaml`):

```yaml
monitoring:
  rolling_window_days: 30
  min_evaluation_pairs: 7
```

```yaml
  mae:
    enabled: true
    threshold: 6.0      # retrain if MAE > 6.0 µg/m³
  psi:
    enabled: true
    threshold: 0.2      # retrain if PSI > 0.2
    bins: 10
```

---

6. Deployment และ Prediction Mode

6.1 เหตุผลที่เลือก Batch Prediction (Hourly)

ระบบออกแบบเป็น **Batch Prediction แบบ Periodic (ทุก 1 ชั่วโมง)** แทน real-time streaming เพราะ:

1. **Use case ไม่ต้องการ sub-second latency** — ผู้ใช้ตัดสินใจล่วงหน้า 24 ชั่วโมง ไม่ใช่ตัดสินใจภายใน milliseconds
2. **PM2.5 เปลี่ยนแปลงช้า** — ค่าเปลี่ยนแปลงอย่างมีนัยสำคัญในระดับชั่วโมง ไม่ใช่วินาที
3. **ประหยัด compute** — pre-compute ผล + cache ไว้ดีกว่า run model ทุก request
4. **ง่ายต่อ monitoring** — prediction แต่ละ batch มี timestamp → ติดตาม performance ได้ชัดเจน

6.2 Prediction Flow

```
[ทุก 1 ชั่วโมง — Airflow Trigger]
        |
        ▼
  Fetch latest 168h data
  (PM2.5, Temp, Wind, Humidity)
        |
        ▼
  Feature Engineering
  (lag, rolling, time features)
        |
        ▼
  Load Champion Model
  from MLflow Registry
        |
        ▼
  Predict PM2.5 for next 24h
  + Compute Prediction Interval
  (e.g., Quantile Regression / Bootstrapped CI)
        |
        ▼
  Store predictions in DB
  (timestamp, station_id, predicted_pm25,
   lower_bound, upper_bound)
        |
        ▼
  Alert Engine checks:
  predicted_pm25 > 75 µg/m³? → Send alert
```

6.3 ความต้องการด้าน Latency

ส่วนประกอบ	Target Latency
Inference (ต่อ batch = 50 สถานี × 24h)	< 30 วินาที
API response (ผู้ใช้ query prediction)	< 500ms
Data fetch จาก AirBKK	< 5 วินาที
Monitoring update	< 1 นาที

6.4 FastAPI Endpoints (Implemented)

ระบบมี REST API จริงผ่าน `src/api.py`:

Endpoint	Method	ทำอะไร
`/health`	GET	Liveness check
`/model/info`	GET	ชื่อโมเดลและ feature list
`/predict`	POST	รับ ≥15 วัน history → ทำนาย PM2.5 วันถัดไป + log prediction
`/actual`	POST	บันทึกค่าจริง → คำนวณ absolute error กับ prediction ที่ log ไว้
`/retrain`	POST	ตรวจ MAE บน rolling window → trigger Airflow DAG ถ้า MAE > threshold

6.5 ข้อจำกัดเมื่อจำนวนผู้ใช้เพิ่มขึ้น

Bottleneck	รายละเอียด	แนวทางแก้ (อนาคต)
API scaling	FastAPI deploy บน single container ยังไม่มี load balancer	Horizontal scaling + Redis cache
Prediction freshness	ถ้า Airflow batch ช้า prediction อาจล้าช้าขึ้น	Async worker + queue
Storage	predictions_log.csv grows unbounded	Retention policy (เก็บ 90 วัน) + migrate to DB
Multi-station	API ปัจจุบัน serve สถานีเดียว (10T)	Parameterize station_id ใน endpoint

---

7. การปรับปรุงจาก Progress 1

ตามข้อเสนอแนะของอาจารย์ใน feedback-projectprogress1.pdf ทีมได้ดำเนินการปรับปรุงดังนี้:

ข้อเสนอแนะ	การปรับปรุง	หลักฐาน
Baseline definition สับสน (Persistence vs Linear Regression ปะปนกัน)	ชี้แจงชัดเจนว่า Baseline 1 = Persistence Model (y_hat = y_t), Baseline 2 = Linear Regression; Progress 2 ใช้ Random Forest เป็น baseline เชิงระบบสำหรับเปรียบเทียบกับ LSTM	Section 1 (model selection rationale), Section 2.2
Forecast Horizon ยังไม่ชัดเจน	กำหนด forecast horizon =**24 ชั่วโมงล่วงหน้า (hourly)** อย่างชัดเจน และ input window = 168 ชั่วโมง (7 วัน) สำหรับ LSTM	Section 1, Section 2.2
ยังไม่มี Confidence Interval / Uncertainty Estimation	เพิ่ม Prediction Interval ใน inference pipeline โดยใช้ Bootstrapped CI หรือ Quantile Regression; แสดง lower/upper bound พร้อม prediction	Section 2.1 (Inference Pipeline), Section 6.2
ยังไม่มี Error Analysis แยกตาม Pollution Regime	กำหนด evaluation breakdown 3 ช่วง: Low (< 35), Moderate (35–75), High (> 75 μg/m³); track MAE_high_regime เป็น monitoring metric หลัก	Section 2.2, Section 5.2
ยังไม่มี Drift Monitoring Plan ชัดเจน	ออกแบบ monitoring layer ครบ: Rolling MAE tracker, PSI-based drift detection, retrain threshold, Airflow DAG สำหรับ automated retrain	Section 3 (ML Lifecycle), Section 5
XGBoost Performance ต่ำกว่าคาด — แนะนำให้ investigate	Train LSTM (PyTorch) และเปรียบเทียบกับครบทุกโมเดล ผลจริง: Random Forest (MAE 4.59) ยังดีที่สุด; LSTM (MAE 6.22) แย่กว่า RF เนื่องจาก tabular lag features เหมาะกับ tree-based models มากกว่า; นี่เป็นข้อค้นพบเชิง empirical ที่ตอบคำถามของอาจารย์	Section 1 (model results table), Section 2.2
แนะนำให้ทำ Feature Importance Plot สำหรับ Random Forest	MLflow log model artifacts ทุก run; feature importance สามารถดึงจาก `rf.feature_importances_` ได้ — อยู่ใน roadmap สำหรับ final report	MLflow artifact plan (Section 4.5)
ปรับปรุง Infrastructure (ไม่ได้อยู่ใน feedback แต่เป็น progress ใหม่)	Deploy full Docker Compose stack (Postgres + MLflow + Airflow + FastAPI); implement 2 Airflow DAGs จริง; API มี `/predict`, `/actual`, `/retrain` endpoints	Section 4 (Infrastructure), Section 6.4
ขยายข้อมูล: จาก 1 สถานี → 86 สถานี	scrape ข้อมูลจากAirBKK ได้รายชั่วโมงตั้งแต่ปี 2024–2026 ครอบคลุม **86 สถานี** และ **13 ตัวแปร** (PM2.5, PM10,	Section 1 (Dataset

ข้อเสนอแนะ	การปรับปรุง	หลักฐาน
และจากรายวัน → รายชั่วโมง	Temp, RH, WS, WD, BP, NO2, O3, CO, NO, NOX, RAIN) ทำให้ dataset ขยายจาก ~530 rows เป็น **~21.9 ล้าน rows** (combined) พร้อมทั้ง meteorological variables ที่ต้องการใน Progress 1 แต่ยังไม่มี	description), Section 4.2

ภาคผนวก

A. Technology Stack (ปรับปรุง)

เครื่องมือ	วัตถุประสงค์
Python 3.11 / 3.14	ภาษาหลัก
PyTorch + skorch	LSTM model
scikit-learn	RF, Ridge, LR preprocessing
FastAPI + uvicorn	REST API serving
MLflow (PostgreSQL)	Experiment tracking + Model registry
Apache Airflow	Pipeline orchestration, 2 DAGs
Docker Compose	Full stack orchestration
PostgreSQL 15	Airflow + MLflow backend DB
ONNX + onnxruntime	Model export + fast inference
pandas / numpy	Data manipulation
matplotlib / seaborn	Visualization, EDA
pytest	Unit testing
Git + GitHub	Version control